

E9-205: Machine Learning for Signal Processing

Homework # 1

AMRIT LAL (24608)

M.Tech - SP

Dept. - ECE

Q.1.)

Sol.

$$a) \quad \frac{\partial}{\partial A} \log(|A|) = 2A^{-1} - \text{diag}(A^{-1})$$

$$\frac{\partial}{\partial A} \log(|A|) = \begin{bmatrix} \frac{\partial}{\partial A_{11}} \log(|A|) & \frac{\partial}{\partial A_{22}} \log(|A|) & \dots & \frac{\partial}{\partial A_{1N}} \log(|A|) \\ \frac{\partial}{\partial A_{21}} \log(|A|) & & & \vdots \\ \vdots & & & \vdots \\ \frac{\partial}{\partial A_{N1}} \log(|A|) & \frac{\partial}{\partial A_{N2}} \log(|A|) & \dots & \frac{\partial}{\partial A_{NN}} \log(|A|) \end{bmatrix}$$

for $a \in A$

$$\frac{\partial}{\partial A} \log(|A|) = \text{tr} \left(A^{-1} \frac{\partial A}{\partial a} \right) \dots \dots \dots \textcircled{1}$$

So,

$$\frac{\partial}{\partial A} \log(|A|) = \begin{bmatrix} \text{tr} \left(A^{-1} \frac{\partial A}{\partial A_{11}} \right) & \dots & \text{tr} \left(A^{-1} \frac{\partial A}{\partial A_{1N}} \right) \\ \vdots & & \vdots \\ \text{tr} \left(A^{-1} \frac{\partial A}{\partial A_{N1}} \right) & \dots & \text{tr} \left(A^{-1} \frac{\partial A}{\partial A_{NN}} \right) \end{bmatrix}$$

$$\text{let } \frac{\partial A}{\partial A_{ij}} = \begin{cases} E_{ii} & ; \text{ if } i=j \\ E_{ij} + E_{ji} & ; \text{ if } i \neq j \end{cases} \dots \dots \textcircled{2}$$

where E_{ij} is the matrix of $N \times N$ in which only at position (i,j) 1 is present & rest all are 0's.

$$\frac{\partial}{\partial A} \log(|A|) = \begin{bmatrix} \text{tr} (A^{-1} E_{11}) & \dots & \text{tr} (A^{-1} (E_{1N} + E_{N1})) \\ \vdots & & \vdots \\ \text{tr} (A^{-1} (E_{N1} + E_{1N})) & \dots & \text{tr} (A^{-1} E_{NN}) \end{bmatrix}$$

$$\therefore \bar{A}'(E_{ii}) = \bar{A}' \begin{bmatrix} 0 & & 0 \\ & \ddots & \\ 0 & & 1 \end{bmatrix} \text{ at } (i,i) \text{ place} = \begin{bmatrix} 0 & & 0 \\ & \ddots & \\ 0 & & \bar{A}_{ii}' \end{bmatrix}$$

Similarly;

$$\bar{A}'(E_{ij} + E_{ji}) = \begin{bmatrix} 0 & & 0 \\ & \ddots & \\ 0 & & A_{ij} \end{bmatrix} \quad \text{'1' at } (i,j) \text{ \& } (j,i)$$

$$\text{So, } \frac{\partial}{\partial A} \log(|A|) = \begin{bmatrix} \bar{A}_{11}' & \bar{A}_{12}' + \bar{A}_{21}' & \dots & \bar{A}_{1N}' + \bar{A}_{N1}' \\ \bar{A}_{21}' + \bar{A}_{12}' & \bar{A}_{22}' & \dots & \bar{A}_{2N}' + \bar{A}_{N2}' \\ \vdots & \vdots & \ddots & \vdots \\ 2\bar{A}_{N1}' & 2\bar{A}_{N2}' & \dots & \bar{A}_{NN}' \end{bmatrix}$$

adding & subtracting

by $\text{diag}(\bar{A}')$ to get squared form

$$= \begin{bmatrix} 2\bar{A}_{11}' & 2\bar{A}_{12}' & \dots & 2\bar{A}_{1N}' \\ \vdots & \vdots & \ddots & \vdots \\ 2\bar{A}_{N1}' & 2\bar{A}_{N2}' & \dots & 2\bar{A}_{NN}' \end{bmatrix} - \begin{bmatrix} \bar{A}_{11}' & & 0 \\ & \ddots & \\ 0 & & \bar{A}_{NN}' \end{bmatrix}$$

$$= 2\bar{A}' - \text{diag}(\bar{A}') \quad \text{hence proved.}$$

Proving eqⁿ ① $\frac{\partial}{\partial a} \log(|A|) = \text{tr}(\bar{A}' \frac{\partial A}{\partial a}) \quad a \in A \text{ i.e. } a = A_{ij}$

$A \rightarrow \text{symmetric}$

$$A = XDX^T$$

$X \rightarrow \text{Orthogonal matrix}$

$D \rightarrow \text{Diagonal matrix}$

$$\frac{\partial}{\partial A} \log(|A|) = \frac{\partial}{\partial A_{ij}} \log(|XDX^T|) = \frac{\partial |D|}{\partial A_{ij}}$$

$$\therefore XX^T = I$$

$$\therefore D = \begin{bmatrix} \lambda_1 & & \\ & \ddots & \\ & & \lambda_N \end{bmatrix} \Rightarrow |D| = \lambda_1 \cdot \lambda_2 \dots \lambda_N$$

$$\Rightarrow \frac{\partial}{\partial A_{ij}} \log(\lambda_1 \cdot \lambda_2 \dots \lambda_N) = \frac{\partial}{\partial A_{ij}} (\log \lambda_1 + \log \lambda_2 + \dots + \log \lambda_N)$$

$$= \frac{\partial}{\partial A_{ij}} \sum_{p=1}^N \log \lambda_p = \sum_{p=1}^N \frac{\partial \log \lambda_p}{\partial A_{ij}} = \sum_{p=1}^N \frac{1}{\lambda_p} \cdot \frac{\partial \lambda_p}{\partial A_{ij}}$$

$\therefore \frac{1}{\lambda_p}$ is element of D^{-1}

$$\begin{aligned} \Rightarrow \frac{\partial}{\partial A} \log(|A|) &= \text{tr} \left(D^{-1} \cdot \frac{\partial D}{\partial A_{ij}} \right) = \text{tr} \left(\frac{D^{-1} \partial D}{\partial A_{ij}} + \frac{\partial X X^T}{\partial A_{ij}} \right) \\ &= \text{tr} \left(X D^{-1} X^T \frac{\partial (X D^{-1} X^T)}{\partial A_{ij}} \right) = \text{tr} \left(A^{-1} \frac{\partial A}{\partial A_{ij}} \right) \end{aligned}$$

hence proved.

Explain eqⁿ (2) :

$$\frac{\partial A}{\partial A_{ij}} = \begin{cases} E_{ii} & \text{if } i=j \\ E_{ij} + E_{ji} & \text{if } i \neq j \end{cases}$$

case-I (when $i=j$)

$$\frac{\partial A}{\partial A_{ii}} = \begin{bmatrix} \frac{\partial A_{11}}{\partial A_{ii}} & \frac{\partial A_{12}}{\partial A_{ii}} & \dots & \frac{\partial A_{1N}}{\partial A_{ii}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial A_{N1}}{\partial A_{ii}} & \frac{\partial A_{N2}}{\partial A_{ii}} & \dots & \frac{\partial A_{NN}}{\partial A_{ii}} \end{bmatrix} = \begin{bmatrix} \frac{\partial A_{11}}{\partial A_{ii}} & & & 0 \\ & \ddots & & \\ 0 & & \ddots & \\ & & & \frac{\partial A_{NN}}{\partial A_{ii}} \end{bmatrix}$$

only diagonal element remains & all other zero.

$$\text{so } \frac{\partial A}{\partial A_{ii}} = E_{ii}$$

$$\text{similarly when } i \neq j \quad \frac{\partial A}{\partial A_{ij}} = E_{ij} + E_{ji}$$

as $A_{ij} = A_{ji}$ (A-symm.) only 2-element will retain
all other will be zero.

$$b.) \quad \frac{\partial}{\partial A} \text{tr}(AB) = 2B - \text{diag}(B)$$

$$\frac{\partial}{\partial A} \text{tr}(AB) = \begin{bmatrix} \frac{\partial \text{tr}(AB)}{\partial A_{11}} & \frac{\partial \text{tr}(AB)}{\partial A_{12}} & \dots & \frac{\partial \text{tr}(AB)}{\partial A_{1N}} \\ \vdots & \vdots & & \vdots \\ \frac{\partial \text{tr}(AB)}{\partial A_{N1}} & \frac{\partial \text{tr}(AB)}{\partial A_{N2}} & \dots & \frac{\partial \text{tr}(AB)}{\partial A_{NN}} \end{bmatrix}$$

$$\therefore \text{tr}(AB) = \sum_{i=1}^N A_{1i} B_{i1} + \sum_{i=1}^N A_{2i} B_{i2} + \dots + \sum_{i=1}^N A_{Ni} B_{iN}$$

given A & B are symmetric

$$\frac{\partial}{\partial A} \text{tr}(AB) = \begin{bmatrix} \frac{\partial A_{11} B_{11}}{\partial A_{11}} & \frac{\partial A_{12} B_{21}}{\partial A_{12}} + \frac{\partial A_{21} B_{12}}{\partial A_{12}} & \dots & \dots \\ \vdots & \vdots & & \vdots \\ \frac{\partial A_{N1} B_{1N}}{\partial A_{N1}} + \frac{\partial A_{1N} B_{N1}}{\partial A_{N1}} & \dots & \dots & \frac{\partial A_{NN} B_{NN}}{\partial A_{NN}} \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} B_{11} & B_{12} + B_{21} & \dots & B_{1N} + B_{N1} \\ \vdots & \vdots & & \vdots \\ B_{1N} + B_{N1} & B_{2N} + B_{N2} & \dots & B_{NN} \end{bmatrix} = \begin{bmatrix} B_{11} & 2B_{12} & \dots & 2B_{1N} \\ 2B_{21} & B_{22} & \dots & 2B_{2N} \\ \vdots & \vdots & & \vdots \\ 2B_{N1} & 2B_{N2} & \dots & B_{NN} \end{bmatrix}$$

we can write above as

$$\Rightarrow \begin{bmatrix} 2B_{11} & 2B_{12} & \dots & 2B_{1N} \\ \vdots & \vdots & & \vdots \\ 2B_{N1} & 2B_{N2} & \dots & 2B_{NN} \end{bmatrix} - \begin{bmatrix} B_{11} & & & 0 \\ & B_{22} & & \\ 0 & & \ddots & \\ & & & B_{NN} \end{bmatrix}$$

$$= 2B - \text{diag}(B) = \text{R.H.S.} \quad (\text{hence proved})$$

Q.02.)

N images each of dim (D)

$N \ll D$

whitening process

$$y = \Lambda^{-1/2} W^T (x - \mu)$$

$x \rightarrow$ i/p D dim image

$\mu \rightarrow$ sample mean

$W \rightarrow$ PCA projection matrix ($D \times d$)

$\Lambda \rightarrow$ diagonal matrix ($d \times d$)

a)

$$\Sigma_T^y = \frac{1}{N} \sum_{n=1}^N y_n y_n^T$$

show $\Sigma_T^y = I \leftarrow (d \times d)$

Sol.

$$\therefore y = \Lambda^{-1/2} W^T (x - \mu)$$

$$\Rightarrow \Sigma_T^y = \frac{1}{N} \sum_{n=1}^N (\Lambda^{-1/2} W^T (x - \mu)) (\Lambda^{-1/2} W^T (x - \mu))^T$$

$$\Rightarrow \Sigma_T^y = \frac{1}{N} \sum_{n=1}^N (\Lambda^{-1/2} W^T (x - \mu)) ((x - \mu)^T W (\Lambda^{-1/2})^T)$$

$$\therefore \Lambda^T = \Lambda$$

$$\Rightarrow \Sigma_T^y = \frac{1}{N} \sum_{n=1}^N (\Lambda^{-1/2} W^T (x - \mu)) ((x - \mu) W \Lambda^{-1/2})$$

$$= \frac{1}{N} \sum_{n=1}^N (\Lambda^{-1/2} W^T (x - \mu) (x - \mu)^T W \Lambda^{-1/2})$$

$\therefore \Lambda^{-1/2} W^T$ & $W \Lambda^{-1/2}$ is independent of n

$$\Rightarrow \Sigma_T^y = \Lambda^{-1/2} W^T \left(\frac{1}{N} \sum_{n=1}^N (x - \mu) (x - \mu)^T \right) W \Lambda^{-1/2}$$

Σ_x (Data covariance matrix)

$$\Rightarrow \Sigma_T^y = \Lambda^{-1/2} W^T \Sigma_x W \Lambda^{-1/2}$$

$\therefore \Sigma_x$'s eigen decomposition $\rightarrow \Sigma_x = W \Lambda W^T$ & $W^T W = I$

$$\Rightarrow \Sigma_T^y = \Lambda^{-1/2} W^T W \Lambda W^T W \Lambda^{-1/2} = \Lambda^{-1/2} \Lambda \Lambda^{-1/2}$$

$$\Rightarrow \boxed{\Sigma_T^y = I}$$

hence proved

b.) Σ_w^Y is invertible.

Sol.

we know for $d=1$

$$J(w) = \frac{w^T \Sigma_B^Y w}{w^T \Sigma_w^Y w}$$

where, $J(w)$: fisher ratio (which needs to be max in LDA)

Σ_B^Y : between class covariance matrix

Σ_w^Y : within class covariance matrix

maximize w^*

$$w^* = \arg \max_w w^T \Sigma_B^Y w \quad \text{s.t.} \quad w^T \Sigma_w^Y w = 1$$

using lagrange method

$$\begin{aligned} \mathcal{L}(w, \lambda) &= w^T \Sigma_B^Y w + \lambda (1 - w^T \Sigma_w^Y w) \\ &= w^T (\Sigma_T^Y - \Sigma_w^Y) w + \lambda (1 - w^T \Sigma_w^Y w) \end{aligned}$$

$$\therefore \Sigma_T^Y = I$$

$$\therefore \Sigma_B^Y = \Sigma_T^Y - \Sigma_w^Y$$

$$\begin{aligned} \Rightarrow \mathcal{L}(w, \lambda) &= w^T (I - \Sigma_w^Y) w + \lambda (1 - w^T \Sigma_w^Y w) \\ &= w^T w - w^T \Sigma_w^Y w + \lambda - \lambda w^T \Sigma_w^Y w \end{aligned}$$

diff. above w.r.t. 'w'

$$\frac{\partial \mathcal{L}(w, \lambda)}{\partial w} = 2w - 2(\Sigma_w^Y w)(1 + \lambda) = 0$$

$$\Rightarrow w = (\Sigma_w^Y w)(1 + \lambda) \Rightarrow \Sigma_w^Y w = \frac{w}{1 + \lambda}$$

$$\text{Let } \mathcal{Q} = \frac{1}{1 + \lambda}$$

$$\text{So } \Sigma_w^Y w = \mathcal{Q} w$$

w is the Evec of Σ_w^Y & $\mathcal{Q}$ is corresponding Eval.

max fisher ratio

$$\begin{aligned} J(w) &= \frac{w^T \Sigma_B^Y w}{w^T \Sigma_w^Y w} = \frac{w^T (\Sigma_T^Y - \Sigma_w^Y) w}{w^T \mathcal{Q} w} \\ &= \frac{w^T w}{w^T \mathcal{Q} w} - \frac{w^T \Sigma_w^Y w}{w^T \mathcal{Q} w} = \frac{w^T w}{w^T \mathcal{Q} w} - 1 = \frac{w^T w}{\mathcal{Q} w^T w} - 1 \end{aligned}$$

$$\Rightarrow J(w) = \frac{1}{\mathcal{Q}} - 1 \quad \Rightarrow J(w) \propto \frac{1}{\mathcal{Q}}$$

$\Rightarrow$ as Q will decrease $J(w)$ will increase

i.e. for $J(w)$ to be $\max^m$ Q should be $\min^m$ as
inverse proportion relation exists b/w them

$\rightarrow$ the Eval corresponding to Evec of Δ_w^y should have $\min^m$ value.

Q.03.)

Sol.

$$x_i ; i=1, 2, \dots, N$$

$$y_i ; i=1, 2, \dots, N$$

linear model

$$y = Ax + b + \epsilon$$

$$\text{noise } \epsilon \sim N(0, \sigma^2 I)$$

$$(A, b, \sigma)$$

$$1) \quad A \in \mathbb{R}^{m \times n} \quad \& \quad x \in \mathbb{R}^n \quad f: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$$

$$f(A) = x^T A^T A x$$

$$\frac{\partial f}{\partial A} = ?$$

$$f(A) = \sum_{i=1}^m ((Ax)_i)^2 = \sum_{i=1}^m \left(\sum_{s=1}^n a_{is} x_s \right)^2$$

$$\frac{\partial f}{\partial a_{pq}} = 2 \left(\sum_{s=1}^n a_{ps} x_s \right) x_q = 2(Ax)_p x_q$$

$$\Rightarrow \frac{\partial f}{\partial A} = 2Ax x^T$$

$$2) \quad A \in \mathbb{R}^{m \times n}, \quad b \in \mathbb{R}^m, \quad x \in \mathbb{R}^n \quad f: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$$

$$f(A) = b^T A x = \sum_{i=1}^m b_i (Ax)_i = \sum_{i=1}^m b_i \sum_{s=1}^n a_{is} x_s$$

$$= \sum_{i=1}^m \sum_{s=1}^n a_{is} b_i x_s$$

$$\frac{\partial f}{\partial a_{pq}} = b_q x_q \quad \Rightarrow \quad \frac{\partial f}{\partial A} = b x^T$$

$$\text{Now, given } y_n = Ax_n + b + \epsilon_n$$

Assume $x_n \in \mathbb{R}^1$, $A \in \mathbb{R}^{M \times D}$, $b \in \mathbb{R}^M$, $\epsilon \in \mathbb{R}^M$ & $y_n \in \mathbb{R}^M$
 $y_n | x_n \sim N(Ax_n + b, \sigma^2 I_m)$

distribution of y_n
 conditioned on x_n

covariance matrix $\Sigma = \sigma^2 I = \begin{bmatrix} \sigma^2 & & 0 \\ & \ddots & \\ 0 & & \sigma^2 \end{bmatrix}$

determinant $\Rightarrow |\Sigma| = \sigma^{2M}$

inverse $\Rightarrow \Sigma^{-1} = \frac{1}{\sigma^2} I$

likelihood for N data points

$$\mathcal{L}(A, b, \sigma) = \prod_{n=1}^N P(y_n | x_n; A, b, \sigma)$$

$$= \prod_{n=1}^N \frac{1}{(2\pi)^{M/2} |\Sigma|^{1/2}} e^{-\frac{1}{2} (y_n - Ax_n - b)^T \Sigma^{-1} (y_n - Ax_n - b)}$$

$$\therefore \Sigma^{-1} = \frac{1}{\sigma^2} I$$

the log likelihood

$$\phi(A, b, \sigma) = -\log(\mathcal{L}(A, b, \sigma))$$

$$= \sum_{n=1}^N \left(\frac{M}{2} \log(2\pi) + M \log(\sigma) + \frac{(y_n - Ax_n - b)^T (y_n - Ax_n - b)}{2\sigma^2} \right)$$

$$= \sum_{n=1}^N \left(\frac{M}{2} \log(2\pi) + M \log(\sigma) + \frac{y_n^T y_n + x_n^T A^T A x_n + b^T b + 2b^T A x_n - 2y_n^T A x_n - 2b^T y_n}{2\sigma^2} \right)$$

$$\Rightarrow \frac{\partial \phi}{\partial A}(A, b, \sigma) = \sum_{n=1}^N \frac{2Ax_n x_n^T + 2bx_n^T - 2y_n x_n^T}{2\sigma^2} = 0 \quad \dots \textcircled{1}$$

$$\& \frac{\partial \phi}{\partial b}(A, b, \sigma) = \sum_{n=1}^N \frac{2b + 2Ax_n - 2y_n}{2\sigma^2} = 0 \quad \dots \textcircled{2}$$

$$\& \frac{\partial \phi}{\partial \sigma}(A, b, \sigma) = \sum_{n=1}^N \left(\frac{M}{\sigma} - \frac{(y_n - Ax_n - b)^T (y_n - Ax_n - b)}{\sigma^3} \right) = 0 \quad \dots \textcircled{3}$$

from eq^s ①, ② & ③

$$A_{ML} = \left(\sum_{n=1}^N (y_n - \bar{y}) x_n^T \right) \left(\sum_{n=1}^N (x_n - \bar{x}) x_n^T \right)^{-1}$$

MLE estimate of A

where $\bar{y} = \frac{1}{N} \sum_{n=1}^N y_n$ & $\bar{x} = \frac{1}{N} \sum_{n=1}^N x_n$

$\sum_{n=1}^N (x_n - \bar{x}) x_n^T$ is invertible

the MLE estimate of b will be

$$b_{ML} = \bar{y} - A_{ML} \bar{x}$$

MLE estimate of σ

$$\sigma_{ML} = \sqrt{\frac{1}{MN} \sum (y_n - A_{ML} x_n - b_{ML})^T (y_n - A_{ML} x_n - b_{ML})}$$

Q.04.)

Solⁿ

given K -classes $p(C_k) = \pi_k$ & $p(\Phi/C_k)$

training data $\{\Phi_n, t_n\}$ for $n=1, \dots, N$

$$t_{n,j} = \delta_{j,k}$$

show $\pi_k = \frac{N_k}{N}$

N_k : no. of points belonging to class K

the likelihood for the model

$$\mathcal{L}(\pi_1, \dots, \pi_K) = \prod_{n=1}^N P(\Phi_n, t_n; \pi_1, \dots, \pi_K)$$

$$\therefore P(\Phi_n, t_n; \pi_1, \dots, \pi_K) = P(\Phi_n/t_n) p(t_n)$$

if this sample belongs to class K

then $t_n = [0, \dots, 1, \dots, 0]^T$ where $t_{nk} = 1$
& others $\rightarrow$ zero

$$\Rightarrow P(\Phi_n/t_n) p(t_n) = P(\Phi_n/C_k) p(C_k) = P(\Phi_n/C_k) \pi_k$$

So

$$\Rightarrow P(\Phi_n, t_n; \pi_1, \dots, \pi_K) = \prod_{k=1}^K (P(\Phi_n/C_k) \pi_k)^{t_{nk}}$$

likelihood over all $\mathcal{L}(\pi_1, \dots, \pi_K) = \prod_{n=1}^N \prod_{k=1}^K (P(\Phi_n/C_k) \pi_k)^{t_{nk}}$

Over log likelihood

$$\begin{aligned} \mathcal{J}(\pi_1, \dots, \pi_K) &= \sum_{n=1}^N \sum_{k=1}^K t_{nk} (\log P(\phi_n/c_k) + \log \pi_k) \\ &= \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log(P(\phi_n/c_k)) + \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log(\pi_k) \\ &= \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log(P(\phi_n/c_k)) + \sum_{k=1}^K N_k \log(\pi_k) \end{aligned}$$

max^m above w.r.t. $\pi_1, \dots, \pi_K$ with constraint $\sum_{k=1}^K \pi_k = 1$

By Lagrangian

$$\begin{aligned} \mathcal{L}(\pi_1, \dots, \pi_K, \lambda) &= \sum_{n=1}^N \sum_{k=1}^K t_{nk} \log(P(\phi_n/c_k)) + \sum_{k=1}^K N_k \log(\pi_k) \\ &\quad + \lambda \left(\sum_{k=1}^K \pi_k - 1 \right) \end{aligned}$$

derivative w.r.t π_k

$$\frac{\partial \mathcal{L}}{\partial \pi_k} = \frac{N_k}{\pi_k} - \lambda = 0$$

$$\Rightarrow \pi_k = \frac{N_k}{\lambda}$$

$$\therefore \sum_{k=1}^K \pi_k = 1$$

$$\text{So } \lambda = \sum_{k=1}^K N_k = N$$

$$\text{So } \pi_k = \frac{N_k}{N} \quad \text{for } 1 \leq k \leq K$$

```
In [83]: import matplotlib.pyplot as plt
from matplotlib.pyplot import imread
import numpy as np
import os
import librosa
from matplotlib.colors import ListedColormap
from scipy.signal import get_window
from numpy.fft import fft
import soundfile as sf
```

QUESTION 5

```
In [84]: highest_fisher_score = 0
best_pca_components = 0
train_data_path = "Data/emotion_classification/train/"
test_data_path = "Data/emotion_classification/test/"
```

Load Dataset

```
In [85]: def load_images_from_directory(directory):
    image_data, image_labels = [], []
    for file_name in os.listdir(directory):
        image_vector = np.asarray(imread(os.path.join(directory, file_name))).flatten()
        image_data.append(image_vector)
        image_labels.append(1 if "happy" in file_name else 0)
    return np.array(image_data), np.array(image_labels)
```

Principal Component Analysis (PCA)

```
In [86]: def apply_pca(input_data, num_features):
    total_samples = input_data.shape[0]
    mean_vector = np.mean(input_data, axis=0)
    centered_data = input_data - mean_vector
    covariance_matrix = np.matmul(centered_data, centered_data.T) / total_samples
    eigen_values, eigen_vectors = np.linalg.eig(covariance_matrix)
    sorted_indices = eigen_values.argsort()[::-1]
    principal_components = np.zeros((centered_data.T.shape[0], num_features))

    for idx in range(num_features):
        principal_components[:, idx] = np.matmul(centered_data.T, eigen_vectors[sorted_indices[idx]]) / np.sqrt(np.abs(eigen_values[sorted_indices[idx]]))

    reduced_representation = np.matmul(centered_data, principal_components)
    return mean_vector, principal_components, reduced_representation
```

Linear Discriminant Analysis (LDA)

```
In [87]: def apply_lda(reduced_data, class_labels, num_features):
    positive_class = reduced_data[class_labels == 1]
    negative_class = reduced_data[class_labels == 0]

    mean_positive = np.mean(positive_class, axis=0, keepdims=True)
    mean_negative = np.mean(negative_class, axis=0, keepdims=True)
```

```

between_class_variance = np.matmul((mean_positive - mean_negative).T, (mean_
within_class_variance = np.matmul((positive_class - mean_positive).T, (posit
                                np.matmul((negative_class - mean_negative).T, (negati

eigen_values, eigen_vectors = np.linalg.eig(np.matmul(np.linalg.inv(within_c
sorted_indices = np.argsort(eigen_values)
lda_projection_vector = eigen_vectors[:, sorted_indices[-1]]

fisher_score = (lda_projection_vector.T @ between_class_variance @ lda_proje

global highest_fisher_score, best_pca_components
if fisher_score > highest_fisher_score:
    highest_fisher_score = fisher_score
    best_pca_components = num_features
    print(f"New highest Fisher score ({highest_fisher_score:.3f}) identified

return lda_projection_vector

```

Train the Model

```

In [88]: def train_model(num_features, plot=False):
    image_data, class_labels = load_images_from_directory(train_data_path)
    mean_vector, pca_components, pca_reduced_data = apply_pca(image_data, num_fe
    lda_vector = apply_lda(pca_reduced_data, class_labels, num_features)

    projected_data = np.matmul(pca_reduced_data, lda_vector)

    if plot:
        plt.scatter(projected_data.real, class_labels, c=class_labels, cmap='vir
        plt.xlabel("LDA Projected Data")
        plt.ylabel("Emotion Class Labels")
        plt.title(f"LDA Projection with {num_features} PCA Components")
        plt.show()

    decision_boundary = (np.mean(projected_data[class_labels == 0]) + np.mean(pr
    classification_flag = decision_boundary > np.mean(projected_data[class_label

    return pca_components, mean_vector, lda_vector, decision_boundary, classific

```

Test the Model

```

In [89]: def evaluate_model(mean_vector, pca_components, lda_vector, decision_boundary, c
    test_data, test_labels = load_images_from_directory(test_data_path)
    centered_test_data = test_data - mean_vector
    pca_test_data = np.matmul(centered_test_data, pca_components)
    lda_projected_data = np.matmul(pca_test_data, lda_vector)

    predicted_labels = [(1 if (value - decision_boundary >= 0) == classification
    model_accuracy = sum([1 for predicted, actual in zip(predicted_labels, test_

    print(f"Model Test Accuracy: {model_accuracy:.2%}\n")

    for num_features in range(1, 19):
        print(f"Training model with {num_features} PCA components")
        plot_flag = True if num_features == 18 else False
        pca_components, mean_vector, lda_vector, decision_boundary, classification_f
        evaluate_model(mean_vector, pca_components, lda_vector, decision_boundary, c

```

```
print(f"Optimal PCA Components: {best_pca_components}, Highest Fisher Score: {hi
```

Training model with 1 PCA components
New highest Fisher score (0.007) identified with 1 components
Model Test Accuracy: 60.00%

Training model with 2 PCA components
New highest Fisher score (0.017) identified with 2 components
Model Test Accuracy: 70.00%

Training model with 3 PCA components
New highest Fisher score (0.025) identified with 3 components
Model Test Accuracy: 60.00%

Training model with 4 PCA components
New highest Fisher score (0.030) identified with 4 components
Model Test Accuracy: 80.00%

Training model with 5 PCA components
New highest Fisher score (0.035) identified with 5 components
Model Test Accuracy: 80.00%

Training model with 6 PCA components
New highest Fisher score (0.035) identified with 6 components
Model Test Accuracy: 80.00%

Training model with 7 PCA components
New highest Fisher score (0.035+0.000j) identified with 7 components
Model Test Accuracy: 80.00%

Training model with 8 PCA components
New highest Fisher score (0.088+0.000j) identified with 8 components
Model Test Accuracy: 70.00%

Training model with 9 PCA components
New highest Fisher score (0.149+0.000j) identified with 9 components
Model Test Accuracy: 80.00%

Training model with 10 PCA components
New highest Fisher score (0.150+0.000j) identified with 10 components
Model Test Accuracy: 80.00%

Training model with 11 PCA components
New highest Fisher score (0.151+0.000j) identified with 11 components
Model Test Accuracy: 70.00%

Training model with 12 PCA components
New highest Fisher score (0.157+0.000j) identified with 12 components
Model Test Accuracy: 80.00%

Training model with 13 PCA components
New highest Fisher score (0.379+0.000j) identified with 13 components
Model Test Accuracy: 80.00%

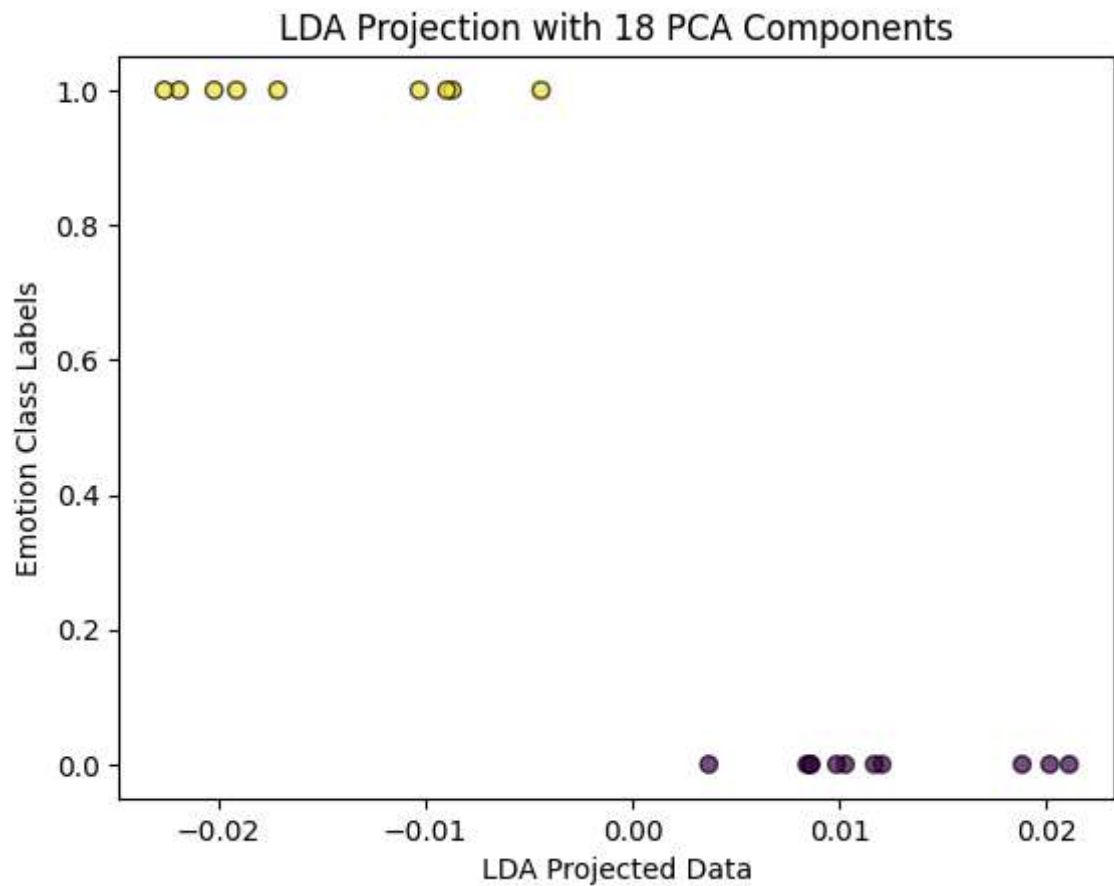
Training model with 14 PCA components
New highest Fisher score (0.398+0.000j) identified with 14 components
Model Test Accuracy: 80.00%

Training model with 15 PCA components
New highest Fisher score (0.653+0.000j) identified with 15 components
Model Test Accuracy: 70.00%

Training model with 16 PCA components
 New highest Fisher score ($0.901+0.000j$) identified with 16 components
 Model Test Accuracy: 90.00%

Training model with 17 PCA components
 New highest Fisher score ($1.046+0.000j$) identified with 17 components
 Model Test Accuracy: 80.00%

Training model with 18 PCA components
 New highest Fisher score ($1.079+0.000j$) identified with 18 components



Model Test Accuracy: 90.00%

Optimal PCA Components: 18, Highest Fisher Score: $1.079+0.000j$

QUESTION 6

PART A

```
In [90]: def load_wav(file_path, rate=16000):
          audio, sr = librosa.load(file_path, sr=rate)
          return audio

          orig_audio = './speechFiles/clean.wav'
          noisy_audio = './speechFiles/noisy.wav'

          orig_signal = load_wav(orig_audio)
          noisy_signal = load_wav(noisy_audio)

          def calc_mel_spec(audio, rate=16000, mel_bins=64, win_ms=20, hop_ms=10):
```

```

fft_size = int(win_ms * rate / 1000)
hop_size = int(hop_ms * rate / 1000)

mel_spec = librosa.feature.melspectrogram(y=audio, sr=rate, n_fft=fft_size,
mel_db = librosa.power_to_db(mel_spec, ref=np.max)

    return mel_db.T
orig_spec = calc_mel_spec(orig_signal)
noisy_spec = calc_mel_spec(noisy_signal)

print(f"Original Spectrogram Shape: {orig_spec.shape}")
print(f"Noisy Spectrogram Shape: {noisy_spec.shape}")

```

Original Spectrogram Shape: (313, 64)

Noisy Spectrogram Shape: (313, 64)

```

In [91]: def calc_whitening_mat(data):
    mean_vec = np.mean(data, axis=0)
    centered_data = data - mean_vec

    cov_mat = np.cov(centered_data, rowvar=False)
    eig_vals, eig_vecs = np.linalg.eigh(cov_mat)
    eig_vals_inv_sqrt = np.diag(1.0 / np.sqrt(eig_vals + 1e-5))
    whiten_mat = eig_vecs @ eig_vals_inv_sqrt @ eig_vecs.T

    return whiten_mat, mean_vec

whiten_mat_orig, mean_orig = calc_whitening_mat(orig_spec)

```

```

In [92]: noisy_centered = noisy_spec - mean_orig
noisy_whitened = noisy_centered @ whiten_mat_orig

cov_noisy_white = np.cov(noisy_whitened, rowvar=False)
print(f"Covariance of Whitened Noisy Spectrogram:\n{cov_noisy_white}")

identity_mat = np.eye(cov_noisy_white.shape[0])
is_identity = np.allclose(cov_noisy_white, identity_mat, atol=0.1)
print(f"Is the covariance matrix close to identity? {is_identity}")

```

Covariance of Whitened Noisy Spectrogram:

```

[[ 9.86104453e-01 -2.33637407e-01 -1.06634903e-01 ... -7.05858129e-02
 -1.08143000e-01  4.41585465e+01]
 [-2.33637407e-01  1.16985591e+00 -1.88830613e-02 ...  1.41257661e-01
 -2.73326204e-01 -1.41157670e+02]
 [-1.06634903e-01 -1.88830613e-02  1.03195726e+00 ... -1.51453779e-01
  6.09389143e-02  1.58101417e+02]
 ...
 [-7.05858129e-02  1.41257661e-01 -1.51453779e-01 ...  1.48440680e+00
 -7.42355354e-01 -5.79515903e+02]
 [-1.08143000e-01 -2.73326204e-01  6.09389143e-02 ... -7.42355354e-01
  1.05247429e+00  7.20912209e+02]
 [ 4.41585465e+01 -1.41157670e+02  1.58101417e+02 ... -5.79515903e+02
  7.20912209e+02  2.02646175e+06]]

```

Is the covariance matrix close to identity? False

```

In [93]: whiten_mat_noisy, mean_noisy = calc_whitening_mat(noisy_spec)

orig_centered = orig_spec - mean_noisy
orig_whitened = orig_centered @ whiten_mat_noisy

```



```

cov_orig_white = np.cov(orig_whitened, rowvar=False)
print(f"Covariance of Whitened Original Spectrogram:\n{cov_orig_white}")

is_identity_reverse = np.allclose(cov_orig_white, identity_mat, atol=0.1)
print(f"Is the covariance matrix (after reversing) close to identity? {is_identi

```

Covariance of Whitened Original Spectrogram:

```

[[ 1.33221397  0.13043057  0.06930949 ...  0.12179907  0.47157783
   0.20427329]
 [ 0.13043057  1.44461872  0.15145888 ...  0.04249241 -0.19436422
  -0.62423574]
 [ 0.06930949  0.15145888  1.34227074 ...  0.12960958  0.10566965
  -0.69897605]
 ...
 [ 0.12179907  0.04249241  0.12960958 ...  1.785819   0.12692554
   0.16755837]
 [ 0.47157783 -0.19436422  0.10566965 ...  0.12692554  7.90364303
  -2.87728544]
 [ 0.20427329 -0.62423574 -0.69897605 ...  0.16755837 -2.87728544
   4.66797034]]

```

Is the covariance matrix (after reversing) close to identity? False

```

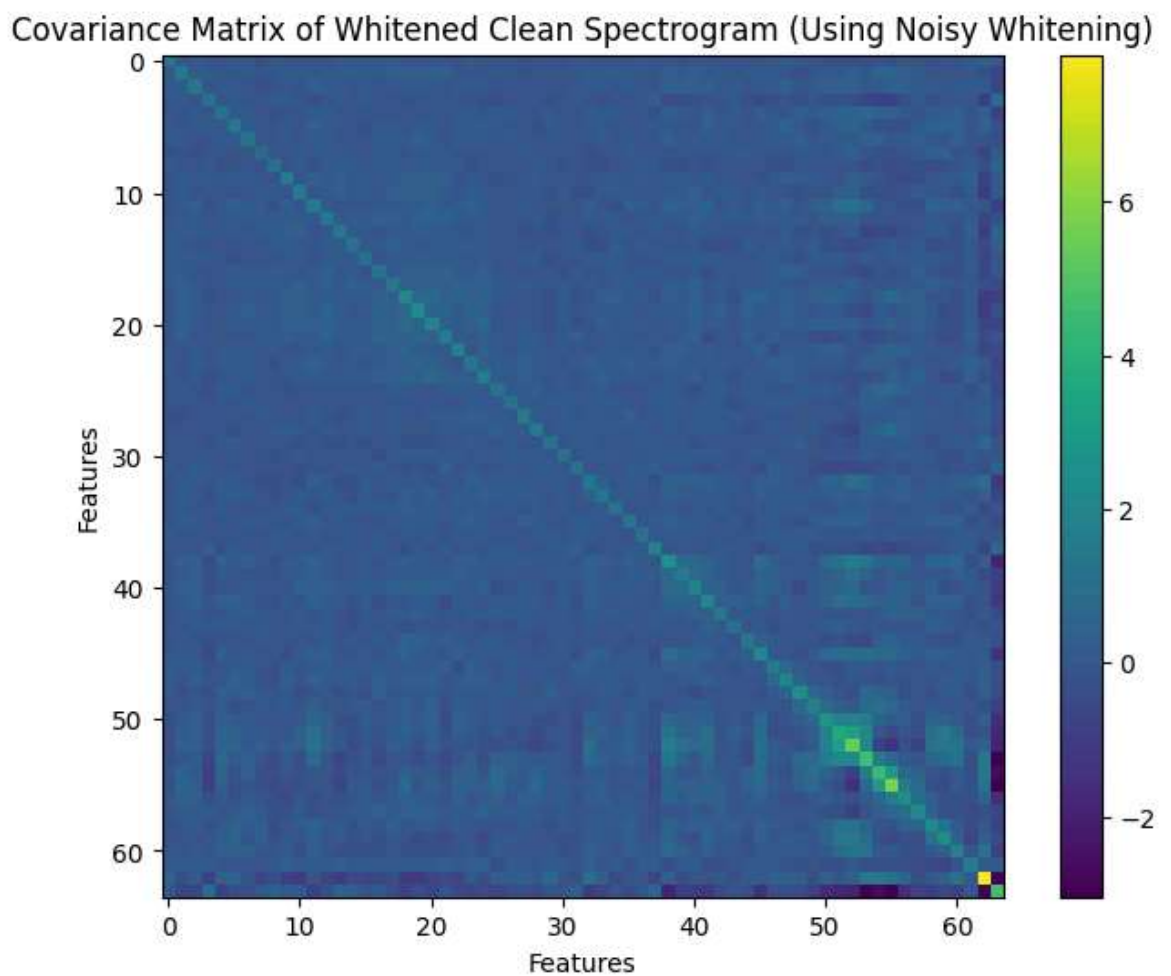
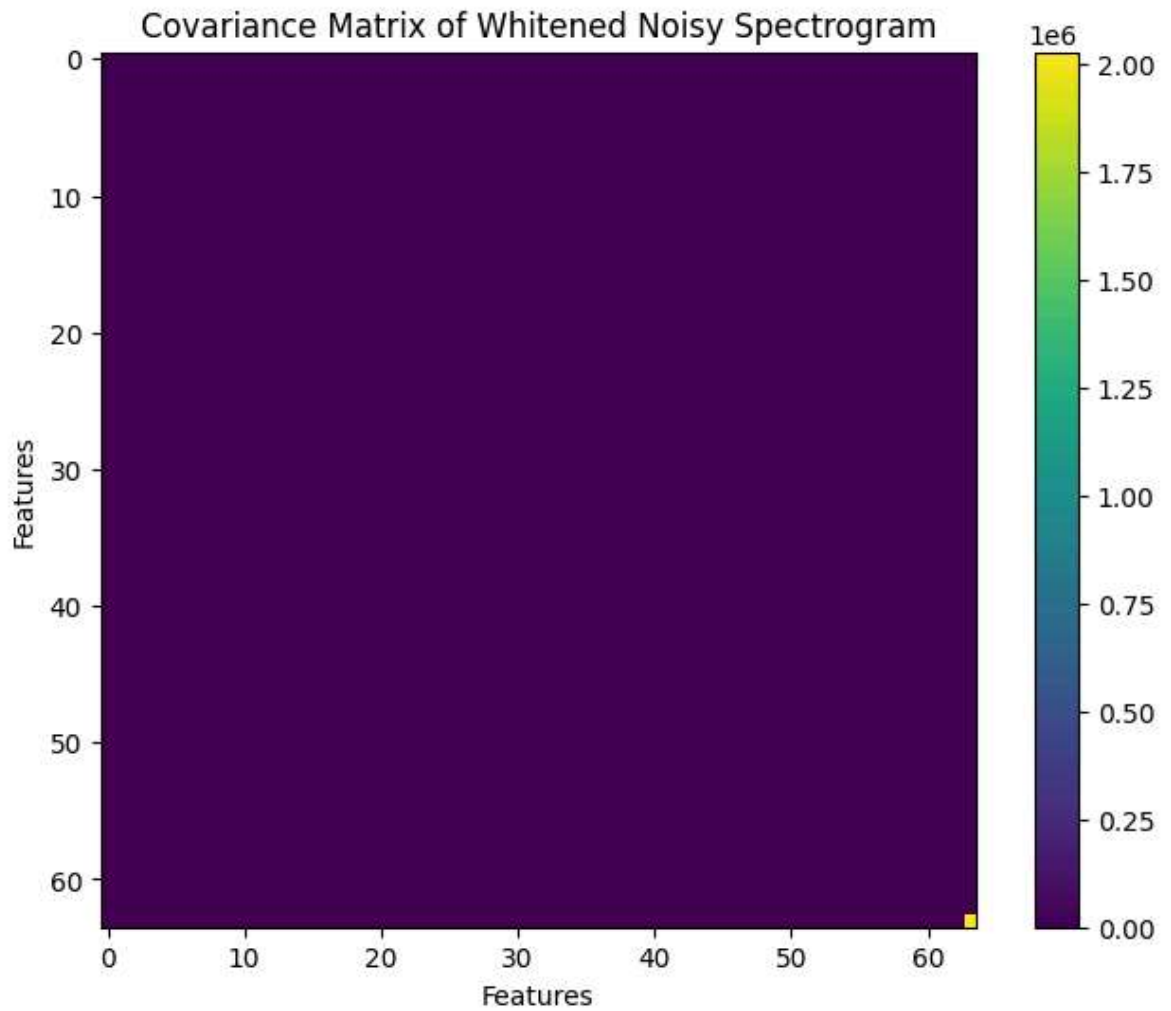
In [94]: def plot_cov_mat(cov_mat, title):
          plt.figure(figsize=(8, 6))
          cmap = plt.cm.viridis
          cmap_colors = cmap(np.linspace(0, 1, 256))
          custom_cmap = ListedColormap(cmap_colors)
          plt.imshow(cov_mat, interpolation='nearest', cmap=custom_cmap)
          plt.colorbar()
          plt.title(title)
          plt.xlabel('Features')
          plt.ylabel('Features')
          plt.grid(False)
          plt.show()

```

```

In [95]: plot_cov_mat(cov_noisy_white, 'Covariance Matrix of Whitened Noisy Spectrogram')
          plot_cov_mat(cov_orig_white, 'Covariance Matrix of Whitened Clean Spectrogram (U

```



PART B

Function to create Mel filter bank

```
In [96]: def create_mel_filters(sampling_rate, fft_points, mel_bands):
    def hz_to_mel_scale(hz_freq):
        return 2595 * np.log10(1 + hz_freq / 700)

    def mel_to_hz_scale(mel_freq):
        return 700 * (10 ** (mel_freq / 2595) - 1)

    mel_scale_points = np.linspace(hz_to_mel_scale(0), hz_to_mel_scale(sampling_rate),
        freq_points = mel_to_hz_scale(mel_scale_points)
    fft_bins = np.floor((fft_points + 1) * freq_points / sampling_rate).astype(int)

    mel_filters = np.zeros((mel_bands, fft_points // 2 + 1))
    for band in range(1, mel_bands + 1):
        mel_filters[band - 1, fft_bins[band - 1]:fft_bins[band]] = np.linspace(0, 1,
        mel_filters[band - 1, fft_bins[band]:fft_bins[band + 1]] = np.linspace(1, 0,
    return mel_filters
```

```
In [97]: # Load audio data
audio_path = './speechFiles/clean.wav'
signal, sample_rate = sf.read(audio_path)

# Parameters for spectrogram
frame_size = int(0.025 * sample_rate)
frame_shift = int(0.010 * sample_rate)
window_function = get_window('hann', frame_size)
num_mel_filters = 40

# Compute the spectrogram
frame_indices = np.arange(0, len(signal) - frame_size, frame_shift)
power_spectrum = np.array([np.abs(fft(signal[idx:idx + frame_size] * window_function))**2 for idx in frame_indices])

# Apply Mel filters
mel_filter_bank = create_mel_filters(sample_rate, frame_size, num_mel_filters)
mel_energy = np.dot(power_spectrum, mel_filter_bank.T)

# Convert to log scale (dB)
log_mel_energy = 10 * np.log10(mel_energy + 1e-10)

# Energy-based feature selection
total_energy = np.sum(log_mel_energy, axis=1)
energy_cutoff = np.percentile(total_energy, 25)
selected_features = log_mel_energy[total_energy > energy_cutoff]

# Normalize features
mean_features = np.mean(selected_features, axis=0)
std_features = np.std(selected_features, axis=0)
normalized_features = (selected_features - mean_features) / (std_features + 1e-10)

# Dimensionality reduction using PCA
cov_matrix = np.cov(normalized_features, rowvar=False)
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)
```

```
principal_components = eigenvectors[:, -10:]
pca_features = normalized_features @ principal_components
```

K-means initialization function

```
In [98]: def initialize_kmeans(data, num_clusters=2, max_iterations=10):
    np.random.seed(42)
    cluster_centers = data[np.random.choice(data.shape[0], num_clusters, replace

    for _ in range(max_iterations):
        distances = np.linalg.norm(data[:, np.newaxis] - cluster_centers, axis=2)
        cluster_assignments = np.argmin(distances, axis=1)
        new_centers = np.array([data[cluster_assignments == j].mean(axis=0) for

        if np.allclose(cluster_centers, new_centers, atol=1e-4):
            break
        cluster_centers = new_centers
    return cluster_centers, cluster_assignments
```

GMM training

```
In [99]: def train_gmm(data, num_components=2, init_type='random', max_iters=20):
    np.random.seed(42)
    samples, features = data.shape

    # Initialize means
    if init_type == 'kmeans':
        means, _ = initialize_kmeans(data, num_components)
    else:
        means = data[np.random.choice(samples, num_components, replace=False)]

    covariances = np.array([np.cov(data.T) + 1e-5 * np.eye(features) for _ in range(num_components)])
    mixing_coefficients = np.ones(num_components) / num_components
    log_likelihood_history = []

    for iteration in range(max_iters):
        # E-step: Calculate responsibilities
        log_probs = np.zeros((samples, num_components))
        for j in range(num_components):
            deviation = data - means[j]
            inv_covariance = np.linalg.inv(covariances[j])
            exponent_term = np.einsum('ij,jk,ik->i', deviation, inv_covariance,
            log_det_cov = np.log(np.linalg.det(covariances[j]) + 1e-10)
            log_probs[:, j] = np.log(mixing_coefficients[j] + 1e-10) - 0.5 * (deviation.T @ inv_covariance @ deviation)

        max_log_probs = np.max(log_probs, axis=1, keepdims=True)
        log_sum_exp = max_log_probs + np.log(np.sum(np.exp(log_probs - max_log_probs), axis=1))
        responsibilities = np.exp(log_probs - log_sum_exp)

        # Compute Log-Likelihood
        total_log_likelihood = np.sum(log_sum_exp)
        log_likelihood_history.append(total_log_likelihood)

        # M-step: Update model parameters
        Nk = responsibilities.sum(axis=0)
        mixing_coefficients = Nk / samples

        for j in range(num_components):
            means[j] = (responsibilities[:, j][:, np.newaxis] * data).sum(axis=0) / Nk
```

```

deviation = data - means[j]
covariances[j] = (responsibilities[:, j][:, np.newaxis, np.newaxis]
                  np.einsum('ij,ik->ijk', deviation, deviation)).sum
covariances[j] += 1e-5 * np.eye(features)

return log_likelihood_history

```

```

In [100... # Train GMM with random and K-means initialization
log_likelihood_random = train_gmm(pca_features, num_components=2, init_type='ran
log_likelihood_kmeans = train_gmm(pca_features, num_components=2, init_type='kme

```

Plotting the log-likelihoods

```

In [101... plt.figure(figsize=(8, 5))
plt.plot(range(1, 21), log_likelihood_random, label='Random Initialization', mar
plt.plot(range(1, 21), log_likelihood_kmeans, label='K-means Initialization', ma

plt.xlabel('Iterations', fontsize=12)
plt.ylabel('Log-Likelihood', fontsize=12)
plt.title('GMM Log-Likelihood Across Iterations', fontsize=14)
plt.legend(fontsize=10)
plt.grid(True, linestyle='--', alpha=0.6)
plt.gca().set_facecolor('black')
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.tight_layout()
plt.show()

```

